# Addition 4: Repairing the Compute Microbenchmark and Re-Evaluating

Real EC2 measurement (not a pure-analysis addition): 5 spot instances launched from the existing shared AMI ( `ami-094f1b962b34950d5` ), benchmarked, and terminated. All 5 instances confirmed terminated after the run — verified via `aws ec2 describe-instances` , zero running/pending instances remain.

## Part 1: The new compute benchmark

`scripts/collection/compute_probe.c` . Design decisions, each made explicit:

- **What it measures**: peak *vectorized* FLOPS (not a serial dependency-chain latency benchmark) — chosen because it's the more representative proxy for what a real query executor's arithmetic-heavy inner loops can achieve, and because TPC-H aggregation (sums, products like `l_extendedprice * (1 - l_discount)` ) is exactly this kind of bulk, vectorizable arithmetic.
- **Anti-elimination**: 8 independent accumulators (exposes ILP for the vectorizer without serializing), each accumulating `data[i]^2` — bounded, no overflow/cancellation risk. A per-outer-iteration perturbation ( `data[it & mask] += 1e-12` ) prevents the compiler from proving the nested loop is invariant across outer iterations and hoisting/eliminating it. A data-dependent checksum (seeded from a runtime argument, not a compile-time constant) is printed and cannot be predicted by the compiler.
- **Cache-resident working set**: 2048 doubles/thread (16KB) — small enough to stay in L1 across the whole run, so this measures compute throughput, not memory bandwidth (kept deliberately separate from the bandwidth signal).
- **Runtime**: self-calibrating — a short probe run determines the iteration count needed to hit a 15-second target, comfortably dwarfing the ~2-5% timing-noise floor established in the original collection protocol.
- **Validation**: reports GFLOPS from the known FLOP count (2 FLOPs/FMA-style step × iterations × array size), with a built-in plausibility gate (flags anything outside [0.3, 300] GFLOPS/thread as suspect). Local validation before ever touching AWS: `-O3 -march=native` gave 12.27 GFLOPS/thread vs. `-O0` 's 2.93 — a believable ~4x vectorization speedup, confirming the loop performs real, non-eliminated work rather than being folded away.
- **Both single- and multi-threaded modes** implemented (pthreads, one thread per core via `nproc` ).

## Part 2: Re-measurement

All 5 machines, single-threaded (pinned via `taskset -c 1` ) and multi-threaded, 15s target each, checksums confirmed non-trivial on every run, all passed the plausibility gate:

| machine | single-thread GFLOPS | multi-thread aggregate GFLOPS | nproc |
|---|---|---|---|
| c5a.xlarge | 10.26 | 40.81 | 4 |
| m5a.xlarge | 8.04 | 31.54 | 4 |
| r5n.large | 12.69 | 24.97 | 2 |
| z1d.large | 15.81 | 28.08 | 2 |
| c7i.large | 13.00 | 25.89 | 2 |

One script hiccup, noted for transparency: the instance-identity self-check via the IMDS endpoint (`curl http://169.254.169.254/...`) returned empty on all 5 machines — this AMI's instances require IMDSv2 (token-based), and the script used a plain IMDSv1-style GET. Machine identity was still independently confirmed via `nproc` matching each instance type's known vCPU count (4/4/2/2/2, correctly matching xlarge/xlarge/large/large/large) and the instance type we explicitly requested at launch. Not a benchmark-validity issue, just a cosmetic logging gap.

### A striking sanity-check finding

| | old benchmark | new (single-thread) |
|---|---|---|
| coefficient of variation across 5 machines | 22.0% | 22.0% |
| rank order (low to high) | m5a < c5a < r5n < c7i < z1d | m5a < c5a < r5n < c7i < z1d |

**The old benchmark's relative differentiation and rank ordering across the 5 machines is essentially identical to the new, rigorously-validated one.** This means the original "simplistic scalar accumulation loop" wasn't fundamentally broken as an instrument — whatever its crudeness, it was capturing approximately the right relative signal. This matters for interpreting what follows: the weak 0.17 correlation was *not* primarily an instrument problem at the single-core level.

## Part 3: Re-evaluation

### Correlation with scaling_factor_p50

| compute signal | pooled corr. (n=420) | compute-labeled-queries corr. (n=60) |
|---|---|---|
| OLD (scalar loop) | 0.168 | 0.306 |
| NEW, single-threaded | 0.213 | 0.378 |
| **NEW, multi-threaded aggregate** | **−0.691** | **−0.816** |

Single-threaded: a modest improvement in magnitude, but **the same wrong sign** as the old benchmark — a positive correlation is physically backwards (it implies more prod compute capacity predicts a *slower* prod, the opposite of bandwidth_ratio's correctly-signed −0.638). This confirms single-core throughput, even measured well, isn't the resource TPC-H scaling actually tracks.

**Multi-threaded aggregate compute is a completely different story**: strong correlation, and critically, the **physically correct sign** — matching bandwidth_ratio's negative direction. More aggregate compute capacity on the prod machine now correctly predicts a *faster* prod (lower scaling factor), exactly as physical intuition demands.

**Plausible mechanism (flagged as a hypothesis, not independently verified here)**: PostgreSQL can execute parallel sequential scans and parallel aggregates across multiple workers for large tables like `lineitem` (6M rows at SF1), if `max_parallel_workers_per_gather` permits. If these queries are in fact running with parallel workers, then a machine's *aggregate* multi-core throughput — not any single core's — is the actual compute resource being consumed, which is exactly what the multi-threaded benchmark measures and the single-threaded one doesn't. This would also explain why instance **core count** (c5a/m5a at 4 vCPU vs r5n/z1d/c7i at 2 vCPU) shows up so strongly in the aggregate metric. **This hypothesis has not been checked against the actual EXPLAIN plans or `postgresql.conf` on the AMI** — confirming or refuting it (e.g., checking for `Gather` / `Parallel Seq Scan` nodes in the query plans) is a natural next step, not something this addition verified.

### Does the gated formula now beat naive-linear?

| compute signal | naive-linear MAPE | gated-formula MAPE | verdict |
|---|---|---|---|
| OLD | 23.03% | 28.06% | does NOT beat |
| NEW single-threaded | 23.03% | 28.38% | does NOT beat |
| **NEW multi-threaded** | 23.03% | **22.30%** | **beats** |

### Full leave-one-machine-out MAPE comparison

| approach | OLD compute | NEW single-thread | NEW multi-thread |
|---|---|---|---|
| naive (=1.0) | 31.97% | 31.97% | 31.97% |
| naive-linear (1/bandwidth_ratio) | 23.03% | 23.03% | 23.03% |
| analytical roofline | 31.14% | 31.98% | **21.60%** |
| bottleneck-gated | 28.06% | 28.38% | **22.30%** |
| learned quantile (p50) | 31.26% | 32.89% | 24.00% |

With the multi-threaded compute signal, **both the analytical roofline and the bottleneck-gated formula now beat naive-linear** — a first for this project.

### Is it statistically real, or noise? (cluster bootstrap, same methodology as Phase 7)

| comparison (naive-linear − X) | point estimate | 95% CI | verdict |
|---|---|---|---|
| vs. analytical (NEW multi-thread) | +1.43pp | [0.30, 2.55] | **SIGNIFICANT** |
| vs. gated (NEW multi-thread) | +0.73pp | [0.28, 1.27] | **SIGNIFICANT** |
| vs. learned quantile (NEW multi-thread) | −0.97pp | [−8.24, 4.27] | not significant |

**This is a genuine, statistically-supported flip, not a lucky point estimate.** For contrast: Phase 7's identical bootstrap test on the *old* compute signal found naive-linear **significantly beat** the analytical formula (95% CI [−11.82, −4.40], entirely on the other side of zero). With the new multi-threaded compute signal, the analytical formula now **significantly beats** naive-linear. The learned GBR model still shows no significant edge either way — consistent with its volatility documented throughout Phases 6/7 and Addition 2.

## Honest verdict

**Fixing the instrument changed the conclusion — but only partially, and only along one specific axis.** Single-core compute throughput, even measured rigorously and shown to be physically plausible, remains a weak and wrong-signed predictor of TPC-H scaling — the original Phase 7 negative result for single-threaded-style compute signals holds up. What changed the outcome was recognizing that **aggregate multi-core compute capacity**, not per-core speed, is the relevant resource — plausibly because these queries can exploit PostgreSQL's parallel query execution against a 6M-row table, though this causal mechanism is a flagged hypothesis, not independently confirmed against the actual query plans here.

Given that, the honest framing is: **compute signal does help, once measured as the right resource (aggregate/multi-core capacity, correlated with instance core count) — the instrument mattered, but the specific fix that worked was choosing the right quantity to measure, not just measuring it more carefully.** The margin is real but modest (21.60% vs 23.03% MAPE, a ~1.4 percentage-point, statistically significant improvement) — not a dramatic accuracy leap, but the first result in this entire project where a physics-informed approach beats the single-ratio baseline with statistical confidence rather than falling short of or merely tying it.

## Updated paper arc

This doesn't overturn Phases 6/7's central deployment recommendation (naive-linear remains simple, robust, and close to the best achievable at this data scale) but it does add a fourth finding to the arc: **compute signal is genuinely useful for cross-environment TPC-H prediction, but only when it captures aggregate/parallel throughput rather than single-core speed** — a specific, falsifiable, mechanistically-grounded claim rather than a vague "the benchmark was bad." Future work: verify the parallel-query hypothesis directly against `EXPLAIN` plans, and consider whether the small remaining margin justifies the added complexity of a two-ratio physics model over the original single-ratio one in a real deployment.

## Artifacts

- `scripts/collection/compute_probe.c` — the validated compute benchmark source.
- `scripts/collection/run_compute_benchmark.sh` — the launch/measure/terminate orchestration script (bash-3.2-compatible).
- `scripts/collection/results/*_compute_benchmark.txt` — raw per-machine benchmark output (all 5 machines, single- and multi-threaded, with checksums).

- `addition4_reevaluate.py` — full re-evaluation script (correlations, gated-formula MAPE, LOMO comparison) for all 3 compute-signal variants.
- `results/addition4_reevaluation.csv` — summary table.